

Ultralyticss

YOLO You Only Look Once

Ultralytics YOLO hujjatlarida YOLO object detection'ni bitta regression masalasi sifatida ko'rib, rasm ichidan bounding box va class probability'larni to'g'ridan-to'g'ri bashorat qilishi tushuntiriladi. Shu sababli YOLO ikki bosqichli detectorlarga qaraganda real-time ishlashga juda mos bo'lgan model oilasi hisoblanadi.

Ultralytics documentation + YOLO documentation bo'yicha hamma asosiy tomonlari: docs tuzilishi, model turlari, tasklar, mode'lar, CLI, Python API, dataset format, training, prediction, validation, export, deployment va YOLOv8 bilan bog'liq amaliy yo'l.

1. Ultralytics nima?

Ultralytics — YOLO modellarini Python package, CLI, dokumentatsiya va deployment ekotizimi sifatida ishlatishga imkon beradigan platforma/kutubxona. Siz ishlatayotgan:

```
pip install ultralytics
```

shundan keyin keladigan:

```
from ultralytics import YOLO
```

aynan Ultralytics package ichidan ishlaydi.

Ultralytics Python documentation'da bu interface object detection, instance segmentation, semantic segmentation va classification kabi vision tasklarni Python projectlarga ulash uchun berilgani aytilgan. Ya'ni siz notebook, .py script, Git Bash, VS Code terminal yoki app

ichidan bitta API bilan model yuklab, train, predict, val, export qila olasiz.

Eng sodda misol:

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
results = model("image.jpg")
```

Bu yerda:

`YOLO("yolov8n.pt")` → pretrained YOLOv8 nano modelni yuklaydi
`model("image.jpg")` → rasmga prediction qiladi

2. YOLO documentation nimani o'rgatadi?

Ultralytics documentation asosan 7 ta katta yo'nalishga bo'linadi:

1. Models → YOLOv8, YOLO11, YOLO12, YOLO26 va boshqa model oilalari
2. Tasks → detection, segmentation, classification, pose, OBB
3. Modes → train, predict, val, export, track, benchmark
4. Datasets → YOLO dataset format, data.yaml, label format
5. Usage → Python API va CLI
6. Guides → model YAML, deployment, optimization, callback va boshqalar
7. Integrations/Export → ONNX, TensorRT, CoreML, OpenVINO va boshqalar

Current Ultralytics docs'da eng yangi model sifatida **YOLO26** ko'p joyda asosiy misol sifatida keltirilgan. Lekin sizning darsingizda **YOLOv8** ishlatilayotgan bo'lsa, asosiy ishlash mantig'i o'xshash: model nomi `yolov8n.pt`, `yolov8m.pt` bo'ladi, commandlar esa `YOLO(...)`, `.predict()`, `.train()`, `.val()`, `.export()` kabi bir xil API bilan ishlaydi. YOLOv8 docs'da ham full documentation uchun Predict, Train, Val va Export sahifalariga qarash kerakligi aytiladi.

3. YOLO nima qiladi?

YOLO computer vision'da odatda **object detection** uchun ishlatiladi. Object detection vazifasi rasm yoki videodagi obyektning:

1. Qaysi class ekanini
2. Qayerda joylashganini
3. Model qanchalik ishonayotganini

aniqlaydi.

Ultralytics object detection docs'da detector output'i obyektlarni o'rab turadigan **bounding boxlar**, **class label** va **confidence score**lardan iborat bo'lishi aytiladi. Bu vazifa obyekt qayerda ekanini topish kerak, lekin obyektning aniq pixel shakli shart bo'lmaganda yaxshi tanlov hisoblanadi.

Masalan, rasmda mashina va odam bo'lsa:

```
person  0.91  box=(x1, y1, x2, y2)
car      0.87  box=(x1, y1, x2, y2)
```

Bu yerda:

person/car → class
0.91/0.87 → confidence
box → bounding box koordinatalari

4. YOLOv8n, YOLOv8m, YOLOv8x nimani bildiradi?

YOLO model nomlarida oxirdagi harf model o'lchamini bildiradi:

n = nano
s = small
m = medium
l = large
x = extra large

Masalan YOLOv8'da:

yolov8n.pt → eng kichik, tezroq, accuracy pastroq
bo'lishi mumkin
yolov8s.pt → small
yolov8m.pt → medium, n'dan aniqroq, lekin sekinroq
yolov8l.pt → large
yolov8x.pt → extra large, og'irroq

Siz notebookda YOLOv8n va YOLOv8m ni taqqoslayotgan bo'lsangiz,
ma'no shunday:

YOLOv8n → tez, yengil, beginner demo uchun yaxshi
YOLOv8m → og'irroq, ko'proq parametr, ko'pincha
yaxshiroq detection

Real hayotda tanlov shunday bo'ladi:

GPU/CPU kuchsiz, tezlik kerak $\rightarrow$ n yoki s
accuracy muhimroq, GPU bor $\rightarrow$ m, l yoki x
real-time webcam kerak $\rightarrow$ n/s
offline analiz kerak $\rightarrow$ m/l/x ham bo'лади

Ultralytics docs'da YOLO modellari real-time inference, pretrained model, yuqori accuracy va ONNX/TensorRT kabi formatlarga export qilish moslashuvchanligini asosiy afzallik sifatida ko'rsatadi.

5. Ultralytics'da asosiy tasklar

Ultralytics YOLO faqat bitta detection modeli emas. Documentation bo'yicha YOLO modellari bir nechta vision tasklarni bajara oladi: object detection, instance segmentation, semantic segmentation, image classification, pose estimation va oriented object detection. Models documentation'da ham YOLO modellari shu tasklar uchun ishlatilishi aytilgan.

5.1. Object Detection

Vazifa:

obyektni box bilan topish

Misol:

person, car, bus, truck, dog, cup

Natija:

class + bounding box + confidence

Command:

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
results = model.predict(source="image.jpg",
save=True)
```

5.2. Instance Segmentation

Vazifa:

har bir obyektни alohida mask bilan ajratish

Detection faqat box beradi. Segmentation esa obyektning aniq shaklini pixel bo'yicha chiqaradi.

Model nomi odatda:

```
yolov8n-seg.pt
yolov8m-seg.pt
```

Misol:

```
from ultralytics import YOLO

model = YOLO("yolov8n-seg.pt")
results = model.predict(source="image.jpg",
save=True)
```

5.3. Classification

Vazifa:

butun rasmga bitta class berish

Masalan siz oldingi projectda:

automobile yoki truck

qilganingiz classification edi.

YOLO classification model nomlari odatda:

yolov8n-cls.pt

yolov8m-cls.pt

Misol:

```
from ultralytics import YOLO
```

```
model = YOLO("yolov8n-cls.pt")
```

```
results = model.predict(source="image.jpg")
```

5.4. Pose Estimation

Vazifa:

odam tanasining keypointlarini topish

Masalan:

head, shoulder, elbow, wrist, knee, ankle

Model:

yolov8n-pose.pt

Misol:

```
from ultralytics import YOLO
```

```
model = YOLO("yolov8n-pose.pt")
```

```
results = model.predict(source="person.jpg",  
save=True)
```

5.5. OBB — Oriented Bounding Box

Oddiy detection boxlari to'g'ri to'rtburchak bo'ladi. OBB esa obyekt qiyshaygan bo'lsa ham aylantirilgan box bilan aniqlaydi.

Kerak bo'ladigan joylar:

satellite image
aerial image
ship detection
rotated object
text region
warehouse package

Model nomi:

yolov8n-obb.pt

5.6. Tracking

Tracking videoda obyektни frame'dan frame'ga kuzatadi.

Detection:

har frame'da obyektни topadi

Tracking:

obyektga ID beradi va uni video bo'ylab kuzatadi

Misol:

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
results = model.track(source="video.mp4",
save=True)
```


Ultralytics docs'da mode sifatida track ham ko'rsatiladi; configuration docs'da mode qiymatlari orasida train, val, predict, export, track, benchmark borligi aytilgan.

6. Ultralytics'da asosiy mode'lar

Documentation'da **mode** degani model bilan nima qilmoqchi ekaningizni bildiradi.

train	→ modelni o'qitish
predict	→ yangi rasm/video ustida prediction
val	→ validation/test baholash
export	→ modelni ONNX/TensorRT/CoreML kabi formatga chiqarish
track	→ videoda tracking
benchmark	→ speed/accuracy benchmarking

CLI documentation'da Ultralytics terminal commandlari umumiy sintaksis bilan ishlashi aytiladi:

yolo TASK MODE ARGS

Masalan:

```
yolo detect predict model=yolov8n.pt  
source=image.jpg
```

Bu yerda:

detect	→ task
predict	→ mode
model/source	→ argumentlar

CLI docs'da bu command structure to'g'ridan-to'g'ri yolo TASK MODE ARGS shaklida berilgan.

7. Python API va CLI farqi

Ultralytics'ni ikki xil ishlatasiz:

1. Python API
2. CLI – command line interface

Python API

Notebook yoki .py file ichida ishlatiladi:

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
results = model.predict(source="image.jpg",
save=True)
```

Python API yaxshi tomoni:

kod ichida natijani olib qayta ishlash mumkin
plots, pandas, OpenCV bilan birlashtirish mumkin
custom app qilish oson
Gradio/FastAPI bilan ulash qulay

Ultralytics Python docs'da pretrained model yuklash, yangi model train qilish va image prediction qilish Python API orqali ko'rsatiladi.

CLI

Git Bash yoki terminalda ishlatiladi:

```
yolo detect predict model=yolov8n.pt  
source=image.jpg save=True
```

CLI yaxshi tomoni:

```
tez test qilish  
notebook ochmasdan run qilish  
script yozmasdan train/predict/export qilish
```

CLI docs'da terminal orqali turli tasklarni **yolo** command bilan,
Python code yozmasdan bajarish mumkinligi aytiladi.

8. Install qilish

Eng oddiy install:

```
pip install ultralytics
```

Tekshirish:

```
yolo checks
```

Yoki Python'da:

```
import ultralytics  
ultralytics.checks()
```

Agar notebookda ishlatsangiz:

```
%pip install ultralytics
```

Google Colab'da ko'pincha:

```
!pip install ultralytics
```

ishlatiladi.

9. Prediction — pretrained YOLO bilan rasmda obyekt topish

Sizning hozirgi notebookingizdagi asosiy mavzu ham shunaqa: `custom_images` ichidagi rasmlarni olib, YOLOv8n va YOLOv8m bilan prediction qilish.

Python bilan

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")

results = model.predict(
    source="custom_images",
    conf=0.25,
    save=True
)
```

Bu yerda:

`source="custom_images"` → papkadagi rasmlarni oladi
`conf=0.25` → 25% dan past confidence
predictionlar tashlanadi
`save=True` → box chizilgan natijani
`runs/` papkaga saqlaydi

Predict docs'da `inference/prediction visual data`'dan ma'no chiqarish jarayoni ekani va `predict mode` real-time inference uchun turli data source'larda ishlatilishi aytiladi.

CLI bilan

```
yolo detect predict model=yolov8n.pt  
source=custom_images conf=0.25 save=True
```

Natija odatda shunga o'xshash joyga tushadi:

```
runs/detect/predict/
```

Agar yana predict qilsangiz:

```
runs/detect/predict2/  
runs/detect/predict3/
```

bo'lishi mumkin.

10. Prediction natijasini kod orqali o'qish

results ichida har bir rasm uchun prediction obyektlari bor.

```
from ultralytics import YOLO  
  
model = YOLO("yolov8n.pt")  
results = model.predict("image.jpg")  
  
r = results[0]  
  
print(r.boxes)  
print(r.names)
```

Boxlar bo'yicha:

```
for box in r.boxes:  
    class_id = int(box.cls[0])  
    class_name = r.names[class_id]
```

```
confidence = float(box.conf[0])
xyxy = box.xyxy[0].tolist()

print(class_name, confidence, xyxy)
```

Bu natija:

```
class_name → person, car, truck...
confidence → model ishonchi
xyxy       → [x1, y1, x2, y2]
```

11. YOLOv8n va YOLOv8m taqqoslash

Notebookingizda ikki model taqqoslanadi:

```
model_n = YOLO("yolov8n.pt")
model_m = YOLO("yolov8m.pt")
```

Odatda:

YOLOv8n → tezroq, yengilroq

YOLOv8m → sekinroq, lekin kuchliroq bo'lishi mumkin

Taqqoslashda ko'riladigan narsalar:

1. Qaysi obyektlarni topdi?
2. Confidence qanchalik yuqori?
3. Bitta obyektни qaysi model aniqroq topdi?
4. Box obyektни to'g'ri o'rab oldimi?
5. Kichik obyektlarni qaysi biri yaxshiroq topdi?
6. False positive bormi?
7. Inference time farqi qanday?

Masalan jadval:

image	model	class	confidence
car.jpg	yolov8n	car	0.81
car.jpg	yolov8m	car	0.90
dog.jpg	yolov8n	dog	0.74
dog.jpg	yolov8m	dog	0.88

Bu darsda muhokama qilish uchun juda yaxshi.

12. Confidence threshold

conf model prediction chiqarishi uchun minimal ishonch chegarasi.

```
results = model.predict("image.jpg", conf=0.25)
```

Agar conf past bo'lsa:

ko'proq obyekt topadi
false positive ko'payishi mumkin

Agar conf baland bo'lsa:

faqat ishonchli obyektlarni chiqaradi
lekin ba'zi haqiqiy obyektlar yo'qolishi mumkin

Misol:

```
model.predict("image.jpg", conf=0.10)  
model.predict("image.jpg", conf=0.50)  
model.predict("image.jpg", conf=0.80)
```

Ultralytics configuration docs'da prediction settings ichida confidence threshold, NMS threshold, input size va boshqa sozlamalar inference performance, speed va accuracy'ga ta'sir qilishi aytilgan.

13. IoU va NMS

YOLO bitta obyekt uchun bir nechta box chiqarishi mumkin. NMS — **Non-Maximum Suppression** — oʻxshash boxlardan eng yaxshisini qoldirish uchun ishlatiladi.

Parametr:

```
model.predict("image.jpg", iou=0.45)
```

Oddiy tushuncha:

IoU katta → ikki box bir-biriga juda oʻxshash

NMS → eng yuqori confidence boxni qoldiradi

Agar yaqin turgan obyektlar bittaga qoʻshilib ketayotgan boʻlsa, iou bilan oʻynab koʻrish mumkin. Agar bitta obyektga koʻp box chiqsa, NMS thresholdni tekshirish kerak.

14. Image size — `imgsz`

Prediction yoki trainingda image size:

```
model.predict("image.jpg", imgsz=640)
```

yoki:

```
yolo detect predict model=yolov8n.pt  
source=image.jpg imgsz=640
```

Kichik obyektlar boʻlsa:

```
imgsz=960
```

yoki:

`imgsz=1280`

foydali bo‘lishi mumkin. Lekin:

`imgsz` katta → sekinroq, GPU/RAM ko‘proq
`imgsz` kichik → tezroq, lekin kichik obyektlar
yo‘qolishi mumkin

15. Dataset format — eng muhim qism

Agar siz pretrained YOLO bilan faqat prediction qilsangiz, dataset label kerak emas. Lekin o‘z custom object detection modelingizni train qilmoqchi bo‘lsangiz, dataset format juda muhim.

Ultralytics YOLO format `data.yaml` orqali dataset root, train/val/test image pathlari va class names dictionary’ini aniqlaydi. Docs’da train/val/test pathlar image directory, image pathlar yozilgan `.txt` file yoki list bo‘lishi mumkinligi ko‘rsatilgan.

Typical structure:

```
dataset/  
├── images/  
│   ├── train/  
│   ├── val/  
│   └── test/  
├── labels/  
│   ├── train/  
│   ├── val/  
│   └── test/  
└── data.yaml
```

`data.yaml`:

```
path: dataset
train: images/train
val: images/val
test: images/test
```

```
names:
  0: automobile
  1: truck
```

Label fayl:

```
labels/train/image_001.txt
```

Ichida:

```
0 0.512 0.438 0.230 0.360
1 0.231 0.620 0.180 0.250
```

Format:

```
class x_center y_center width height
```

Ultralytics docs'da har bir image uchun bitta .txt label file bo'lishi, har qatorda `class x_center y_center width height` format bo'lishi, koordinatalar normalized xywh formatda 0 dan 1 gacha bo'lishi kerakligi aytiladi. Pixel koordinatalar bo'lsa, x va width image width'ga, y va height image height'ga bo'linishi kerak. Class number esa 0 dan boshlanishi kerak.

16. YOLO label formatni oddiy misol bilan tushuntirish

Rasm o'lchami:

width = 1000 px
height = 500 px

Obyekt box:

x_center = 500 px
y_center = 250 px
box_width = 200 px
box_height = 100 px

YOLO normalized format:

$x_center = 500 / 1000 = 0.5$
 $y_center = 250 / 500 = 0.5$
 $width = 200 / 1000 = 0.2$
 $height = 100 / 500 = 0.2$

Label:

0 0.5 0.5 0.2 0.2

Agar class 0 automobile bo'lsa:

0 0.5 0.5 0.2 0.2

automobile box degani.

Agar class 1 truck bo'lsa:

1 0.5 0.5 0.2 0.2

truck box degani.

17. Custom dataset bilan training

Custom dataset train qilish uchun:

```
from ultralytics import YOLO
```

```
model = YOLO("yolov8n.pt")
```

```
results = model.train(  
    data="data.yaml",  
    epochs=100,  
    imgsz=640,  
    batch=16  
)
```

CLI bilan:

```
yolo detect train model=yolov8n.pt data=data.yaml  
epochs=100 imgsz=640 batch=16
```

Ultralytics train docs'da training modelga data berib, parametrlarini prediction aniq bo'lishi uchun sozlash jarayoni sifatida tushuntiriladi. Dataset docs'da esa custom dataset bilan training boshlash uchun dataset to'g'ri formatda bo'lishi va YAML file'da path/class names berilishi kerakligi ko'rsatiladi.

Training natijalari odatda:

```
runs/detect/train/
```

ichiga tushadi.

Muhim fayllar:

```
runs/detect/train/weights/best.pt  
runs/detect/train/weights/last.pt  
runs/detect/train/results.png  
runs/detect/train/confusion_matrix.png  
runs/detect/train/results.csv
```

`best.pt` — validation bo'yicha eng yaxshi model. `last.pt` — oxirgi epoch modeli.

18. Training parametrlarini tushunish

Ultralytics configuration docs'da training settings performance, speed va accuracy'ga ta'sir qilishi, batch size, learning rate, momentum, weight decay, optimizer, loss function va dataset composition muhim ekani aytiladi.

Eng ko'p ishlatiladigan parametrlar:

<code>data</code>	→ <code>data.yaml</code> path
<code>epochs</code>	→ training necha marta datasetni ko'radi
<code>imgsz</code>	→ image size
<code>batch</code>	→ batch size
<code>device</code>	→ CPU/GPU tanlash
<code>workers</code>	→ data loading processlari
<code>optimizer</code>	→ SGD, AdamW va boshqalar
<code>lr0</code>	→ boshlang'ich learning rate
<code>patience</code>	→ early stopping
<code>project</code>	→ output qayerga saqlansin
<code>name</code>	→ experiment nomi

Misol:

```
results = model.train(  
    data="data.yaml",  
    epochs=50,  
    imgsz=640,  
    batch=8,  
    device=0,
```

```
    project="runs/custom",  
    name="automobile_truck_yolov8n"  
)
```

Agar CPU bo'lsa:

```
device="cpu"
```

Agar GPU bo'lsa:

```
device=0
```

19. Validation — modelni baholash

Validation mode modelning val/test datasetdagi sifatini o'lchaydi.

Python:

```
from ultralytics import YOLO  
  
model = YOLO("runs/detect/train/weights/best.pt")  
metrics = model.val(data="data.yaml")
```

CLI:

```
yolo detect val  
model=runs/detect/train/weights/best.pt  
data=data.yaml
```

Validation settings docs'da validation model performance'ni validation datasetda baholash uchun ishlatilishi, batch size, validation frequency va performance metrics kabi sozlamalar jarayonga ta'sir qilishi aytiladi.

Object detection'da eng muhim metriclar:

Precision
Recall
mAP50
mAP50-95
Confusion matrix

20. mAP nima?

YOLO object detection'da eng muhim metriclardan biri — **mAP**, ya'ni mean Average Precision.

Oddiy tushuncha:

mAP model obyektini to'g'ri class bilan, to'g'ri joyda topyaptimi, shuni o'lchaydi

mAP50

$\text{IoU} \geq 0.50$ bo'lsa, prediction to'g'ri deb hisoblanadi

Bu yumshoqroq metric.

mAP50-95

IoU 0.50 dan 0.95 gacha har xil thresholdlarda o'rtacha AP

Bu qattiqroq metric. Box aniq bo'lishi kerak.

Agar:

mAP50 yuqori, mAP50-95 past

bo'lsa, model obyektni topyapti, lekin box joylashuvi unchalik aniq emas.

21. Precision va Recall

Precision

Savol:

Model topgan obyektlarning nechitasi haqiqatan to'g'ri?

Agar precision past bo'lsa:

model ko'p false positive qilyapti

Masalan, model har joyni truck deb belgilab yuboryapti.

Recall

Savol:

Haqiqiy obyektlarning nechtasini model topa oldi?

Agar recall past bo'lsa:

model ko'p obyektni o'tkazib yuboryapti

Masalan, rasmda 10 ta truck bor, model 4 tasini topdi.

Taskga qarab qaysi biri muhimligi farq qiladi:

security / defect detection → recall juda muhim
false alarm qimmat bo'lsa → precision juda muhim

22. Export — modelni deploymentga tayyorlash

Trainingdan keyin model `.pt` formatda bo‘ladi:

`best.pt`

Lekin production’da ko‘pincha boshqa formatlar kerak:

ONNX

TensorRT

OpenVINO

CoreML

TFLite / LiteRT

TorchScript

Ultralytics export docs’da export mode trained modelni real-world deployment uchun turli formatlarga chiqarish imkonini berishi, ONNX, TensorRT, CoreML kabi formatlarga export qilish mumkinligi aytiladi. Docs’da TensorRT GPU’da speedup, ONNX/OpenVINO CPU’da speedup berishi ham ko‘rsatilgan.

Python:

```
from ultralytics import YOLO
```

```
model = YOLO("best.pt")  
model.export(format="onnx")
```

CLI:

```
yolo export model=best.pt format=onnx
```

TensorRT:

```
yolo export model=best.pt format=engine
```

OpenVINO:

```
yolo export model=best.pt format=openvino
```

23. Export formatlar qachon kerak?

.pt	→ PyTorch ichida ishlatish
ONNX	→ universal inference, ONNX Runtime
TensorRT	→ NVIDIA GPU'da tez deployment
OpenVINO	→ Intel CPU/GPU uchun
CoreML	→ iPhone/macOS uchun
TFLite	→ mobile/edge device uchun
TorchScript	→ PyTorch production

Agar siz faqat notebook va demo qilsangiz:

.pt yetadi

Agar real app/server/device qilsangiz:

export kerak bo'ladi

24. YOLOv8 pretrained model COCO bilan nimani taniydi?

YOLOv8 detection pretrained modellari odatda COCO dataset obyektlarini taniydi. COCO classlar ichida:

person
bicycle
car
motorcycle
airplane

bus
train
truck
boat
traffic light
...
cat
dog
...
cup
bottle
...

Ultralytics dataset docs'dagi COCO8 example class listida **person**, **bicycle**, **car**, **bus**, **truck**, **cat**, **dog**, **cup**, **bottle** kabi classlar borligi ko'rsatiladi.

Shuning uchun sizning starter image uchun:

person
car
bus
bicycle
dog
cat
cup
bottle
truck

yaxshi ishlashi mumkin.

Lekin pretrained YOLOv8:

uzbek pasport
maxsus zavod detal
o'zingizning private class
juda noaniq chizma
medical maxsus obyekt

kabi narsalarni bilmasligi mumkin. Bunday vazifada custom dataset bilan training kerak.

25. YOLOv8 notebookdagi yaxshi pedagogik workflow

Siz yozayotgan notebook uchun yaxshi tartib:

1. Install and Import
2. Upload Simple Custom Images
3. Check Uploaded Images
4. Load YOLOv8n and YOLOv8m
5. Run Predictions
6. Compare Detected Objects
7. Visualize Bounding Boxes
8. Discuss Errors
9. Summary

Har bir bo'limda bitta aniq maqsad bo'lsin.

Masalan:

1. Install and Import

```
%pip install ultralytics -q
```

```
from ultralytics import YOLO
```

```
from pathlib import Path
from PIL import Image
import pandas as pd
import matplotlib.pyplot as plt
```

2. Image folder

```
CUSTOM_DIR = Path("custom_images")
CUSTOM_DIR.mkdir(exist_ok=True)
```

3. Load models

```
model_n = YOLO("yolov8n.pt")
model_m = YOLO("yolov8m.pt")
```

4. Predict

```
results_n = model_n.predict(source=str(CUSTOM_DIR),
conf=0.25, save=True)
results_m = model_m.predict(source=str(CUSTOM_DIR),
conf=0.25, save=True)
```

5. Extract table

```
rows = []

for model_name, results in [("YOLOv8n", results_n),
 ("YOLOv8m", results_m)]:
    for r in results:
        image_name = Path(r.path).name
        for box in r.bboxes:
            cls_id = int(box.cls[0])
            class_name = r.names[cls_id]
            conf = float(box.conf[0])
```

```
x1, y1, x2, y2 = box.xyxy[0].tolist()

rows.append({
    "image": image_name,
    "model": model_name,
    "class": class_name,
    "confidence": round(conf, 3),
    "x1": round(x1, 1),
    "y1": round(y1, 1),
    "x2": round(x2, 1),
    "y2": round(y2, 1),
})
```

```
df = pd.DataFrame(rows)
df
```

26. Ultralytics Results obyektini tushunish

`model.predict()` qaytargan `results` list bo'ladi. Har bir rasm uchun bitta result.

```
results = model.predict("image.jpg")
r = results[0]
```

Eng ko'p kerak bo'ladigan joylar:

<code>r.path</code>	→ image path
<code>r.names</code>	→ class id dan class name
<code>r.bboxes</code>	→ detection boxes
<code>r.bboxes.cls</code>	→ class ids
<code>r.bboxes.conf</code>	→ confidence scores
<code>r.bboxes.xyxy</code>	→ x1,y1,x2,y2
<code>r.plot()</code>	→ box chizilgan image array

Example:

```
annotated = results[0].plot()

plt.imshow(annotated)
plt.axis("off")
plt.show()
```

27. YOLO output folderlari

Agar:

```
model.predict(source="custom_images", save=True)
```

qilsangiz, output:

```
runs/detect/predict/
```

ichiga saqlanadi.

Agar ikkinchi marta predict qilsangiz:

```
runs/detect/predict2/
```

bo'lishi mumkin.

Agar project/name bersangiz:

```
model.predict(
    source="custom_images",
    save=True,
    project="outputs",
    name="yolov8n_predictions"
)
```

Natija:

outputs/yolov8n_predictions/

ichiga tushadi.

Bu tartib report uchun yaxshi.

28. Custom images bilan xatolar

28.1. Rasm ochilmaydi

Siz oldin duch kelgansiz: .jpeg nomli fayl aslida image emas, HTML bo‘lib qolishi mumkin. YOLO ham bunday faylni ishlata olmaydi.

Tekshirish:

```
from PIL import Image
from pathlib import Path

for path in Path("custom_images").glob("*"):
    try:
        img = Image.open(path)
        print(path.name, img.size, img.mode)
    except Exception as e:
        print(path.name, "ERROR:", e)
```

28.2. Linkdan noto‘g‘ri yuklash

Google share linklar ko‘pincha direct image emas. Python `urlretrieve` bilan yuklaganda HTML sahifa .jpg deb saqlanib qolishi mumkin.

To‘g‘ri link Content-Type: image/jpeg yoki image/png qaytarishi kerak.

28.3. Drawing yoki cartoon yomon ishlashi mumkin

Pretrained YOLO real photo datasetlarda o'qitilgan bo'lgani uchun real rasmlar chizmaga qaraganda yaxshiroq ishlaydi.

28.4. Obyekt juda kichik bo'lsa

Model topmasligi mumkin. `imgsz`ni oshiring:

```
model.predict("image.jpg", imgsz=960)
```

28.5. Confidence past bo'lsa

`conf`ni kamaytirib ko'ring:

```
model.predict("image.jpg", conf=0.10)
```

29. YOLO project uchun yaxshi papka strukturasi

Beginner project uchun:

```
yolo_bounding_box_intro/  
├─ custom_images/  
│   ├── person.jpg  
│   ├── car.jpg  
│   └── dog.jpg  
├─ notebooks/  
│   └── 01_yolov8_bounding_box_intro.ipynb  
├─ outputs/  
│   ├── yolov8n_predictions/  
│   ├── yolov8m_predictions/  
│   └── comparison_table.csv
```

```
|— README.md
|— requirements.txt
```

requirements.txt:

```
ultralytics
pandas
matplotlib
pillow
jupyter
ipywidgets
```

30. CLI bilan beginner notebookdagi ishlarni bajarish

Agar notebook ishlatmasdan Git Bash'da sinamoqchi bo'lsangiz:

```
pip install ultralytics
```

YOLOv8n:

```
yolo detect predict model=yolov8n.pt
source=custom_images conf=0.25 save=True
```

YOLOv8m:

```
yolo detect predict model=yolov8m.pt
source=custom_images conf=0.25 save=True
```

Agar output joyini belgilamoqchi bo'lsangiz:

```
yolo detect predict model=yolov8n.pt
source=custom_images conf=0.25 project=outputs
name=yolov8n_predictions save=True
```

```
yolo detect predict model=yolov8m.pt  
source=custom_images conf=0.25 project=outputs  
name=yolov8m_predictions save=True
```

31. Custom YOLO training uchun annotation toollar

Agar keyin o'zingizning custom detection model qilmoqchi bo'lsangiz, rasmga box chizish kerak bo'ladi.

Tools:

Roboflow

CVAT

LabelImg

Label Studio

makesense.ai

Ultralytics Platform

Annotationdan keyin YOLO format:

```
class x_center y_center width height
```

bo'lishi kerak.

Custom training uchun eng katta qoida:

```
yomon label → yomon model
```

Boxlar to'g'ri, classlar aniq, train/val/test split to'g'ri bo'lishi kerak.

32. YOLO bilan ishlaganda eng ko‘p uchraydigan xatolar

1. Class index 0 dan boshlanmagan

Noto‘g‘ri:

```
1 0.5 0.5 0.2 0.2
2 0.4 0.3 0.1 0.1
```

Agar 2 class bo‘lsa, to‘g‘ri:

```
0 ...
1 ...
```

Docs’da class numbers zero-indexed bo‘lishi kerakligi aytiladi.

2. Pixel coordinate berib qo‘yish

Noto‘g‘ri:

```
0 500 250 200 100
```

To‘g‘ri:

```
0 0.5 0.5 0.2 0.2
```

3. data.yaml path xato

Masalan:

```
train: image/train
```

deb yozib yuborasiz, aslida:

```
train: images/train
```

bo‘lishi kerak.

4. Image va label nomlari mos emas

images/train/car001.jpg

labels/train/car001.txt

mos bo‘lishi kerak.

5. Label yo‘q obyektlar

Rasmda obyekt bor, lekin label qilinmagan bo‘lsa, model chalkashadi.

6. Train va validation aralashib ketgan

Bir xil video frame’lari train va valga tushsa, validation score yolg‘on yuqori chiqadi.

33. YOLOv8 va YOLO26 farqiga ehtiyot bo‘lish

Sizning darsingiz YOLOv8 bilan. Hozirgi Ultralytics docs esa ko‘p joyda YOLO26’ni misol qilib ko‘rsatmoqda. Bu documentation yangilangani uchun. Lekin asosiy API deyarli bir xil:

```
model = YOLO("yolov8n.pt")
model.predict(...)
model.train(...)
model.val(...)
model.export(...)
```

YOLO26 documentation’da yangi model family detection, instance segmentation, semantic segmentation, classification, pose estimation

va OBB tasklarni qo‘llashi, n/s/m/l/x scale variantlari borligi aytiladi. Unda DFL-free regression, end-to-end NMS-free inference, task-specific heads/losses kabi yangi improvementlar sanalgan.

Siz beginner dars uchun YOLOv8 ishlatsangiz ham juda yaxshi, chunki YOLOv8 docs’da simple training/inference examples mavjud va full mode documentation Train/Predict/Val/Export sahifalariga bog‘langan.

34. Documentationni qanday o‘qish kerak?

Siz YOLO docs’ni tartibsiz o‘qisangiz chalkashib ketadi. Eng yaxshi o‘rganish tartibi:

1. Quickstart / Python Usage
2. Tasks → Detect
3. Modes → Predict
4. Results object
5. Datasets → Detect format
6. Modes → Train
7. Modes → Val
8. Modes → Export
9. CLI usage
10. Configuration arguments

Shu tartibda o‘qisangiz, YOLO mantiqi to‘liq joyiga tushadi.

35. Sizning notebookingiz uchun qisqa summary

Sizning “Bounding Box Introduction with YOLOv8” notebookingiz documentation bo‘yicha quyidagilarni o‘rgatishi kerak:

1. YOLO nima?
2. Bounding box nima?
3. Pretrained YOLOv8n va YOLOv8m qanday yuklanadi?
4. custom_images ichidagi rasmlar qanday ko‘riladi?
5. model.predict() qanday ishlaydi?
6. confidence nima?
7. box koordinatalar qanday olinadi?
8. YOLOv8n va YOLOv8m natijalari qanday taqqoslanadi?
9. prediction image’lar qayerga saqlanadi?
10. model qachon xato qiladi?

Bu beginner uchun juda to‘g‘ri yo‘l.

36. Eng kerakli kodlar jamlanmasi

Install

```
%pip install ultralytics pandas matplotlib pillow -q
```

Import

```
from ultralytics import YOLO
from pathlib import Path
```

```
from PIL import Image
import pandas as pd
import matplotlib.pyplot as plt
```

Image folder

```
CUSTOM_DIR = Path("custom_images")
CUSTOM_DIR.mkdir(exist_ok=True)
```

Load models

```
model_n = YOLO("yolov8n.pt")
model_m = YOLO("yolov8m.pt")
```

Predict

```
results_n = model_n.predict(
    source=str(CUSTOM_DIR),
    conf=0.25,
    save=True,
    project="outputs",
    name="yolov8n_predictions"
)
```

```
results_m = model_m.predict(
    source=str(CUSTOM_DIR),
    conf=0.25,
    save=True,
    project="outputs",
    name="yolov8m_predictions"
)
```


Extract table

```
rows = []

for model_name, results in [("YOLOv8n", results_n),
                             ("YOLOv8m", results_m)]:
    for result in results:
        image_name = Path(result.path).name

        for box in result.bboxes:
            cls_id = int(box.cls[0])
            class_name = result.names[cls_id]
            confidence = float(box.conf[0])
            x1, y1, x2, y2 = box.xyxy[0].tolist()

            rows.append({
                "image": image_name,
                "model": model_name,
                "class": class_name,
                "confidence": round(confidence, 3),
                "x1": round(x1, 1),
                "y1": round(y1, 1),
                "x2": round(x2, 1),
                "y2": round(y2, 1),
            })

df = pd.DataFrame(rows)
df
```

Save comparison

```
Path("outputs").mkdir(exist_ok=True)
df.to_csv("outputs/detected_objects_comparison.csv"
, index=False)
```

37. Yakuniy xulosa

Ultralytics documentation sizga YOLO'ni amaliy ishlatish uchun kerak bo'lgan hamma narsani beradi: Python API, CLI, tasklar, mode'lar, dataset format, train, predict, val, export va deployment.

YOLO documentation esa modelning asosiy g'oyasini tushuntiradi: rasmga bir marta qarab, obyekt classlari va bounding boxlarni topish. Original YOLO paper'da bu yondashuv object detection'ni bitta neural network orqali bounding box va class probability'larni full image'dan to'g'ridan-to'g'ri prediction qilish sifatida tushuntirilgan.

hozirgi dars/notebook uchun eng muhim hujjat bo'limlari:

Python Usage	→ from ultralytics import YOLO
CLI	→ yolo TASK MODE ARGS
Predict	→ pretrained model bilan inference
Detect task	→ bounding box, class, confidence
Dataset	→ data.yaml va YOLO label format
Train	→ custom datasetda model o'qitish
Val	→ mAP, precision, recall
Export	→ ONNX, TensorRT, CoreML, OpenVINO

Beginner sifatida eng avval shuni mukammal qiling:

```
custom_images → YOLOv8n predict
custom_images → YOLOv8m predict
natijani jadvalga chiqarish
```

bounding boxlarni koʻrish
confidence farqini tahlil qilish
n va m model farqini yozish

keyingi qadam — **custom dataset annotation + data.yaml + YOLO
train**